Отчёт по А/Б тесту платёжной воронки

Период теста: 1–30 апреля 2024 · 1501 пользователей (A: 751, B: 750) · A — контроль, B — тест

Главный результат. Вариант B значимо повысил конверсию пользователей в успешный платёж: **65,6% → 80,4%** (+14,8 п.п., 95% ДИ разницы [10,3; 19,2] п.п., p < 0,001). Эффект виден на всех шагах воронки и во всех сегментах, время прохождения не ухудшилось. **Рекомендация: раскатить вариант B на всех пользователей.**

1. Постановка задачи

Цель: оценить, влияет ли новый вариант платёжной воронки (группа B) на конверсию в успешный платёж по сравнению с контролем (группа A). Воронка состоит из четырёх шагов: открытие платежа, ввод данных, подтверждение, успешная оплата.

Основная метрика: доля пользователей, у которых есть хотя бы один успешный платёж. Считается на уровне пользователя, а не платежа: у человека в среднем 1,8 платежа, платежи одного пользователя зависимы, и тест по платежам завысил бы значимость. **Гипотезы:** H0 — конверсии в группах равны, H1 — различаются; уровень значимости α = 0,05; критерий — z-тест для двух долей.

Выбор метода. Данные разделены по клиентам, поэтому единица анализа — пользователь. Метрика бинарная, группы независимы, в каждой ячейке таблицы 2×2 не менее 147 наблюдений, поэтому применим z-тест для двух долей (доверительные интервалы по Уилсону). Устойчивость проверена хи-квадратом, бутстрэпом и логистической регрессией с поправкой на возраст, город и устройство. Время прохождения сравнивается тестом Манна — Уитни (дискретное, не нормальное распределение); для трёх вторичных тестов по шагам использована поправка Бонферрони (порог 0,017).

2. Данные и очистка

Исходные данные — два листа: «Пользователи» (1504 строки) и «Платежи» (2661 строка). После очистки и объединения: **1501 пользователей и 2659 платежей**. Исключено менее 0,2% данных:

Что исключено	Причина
Пользователи 9002, 9003	Невозможный возраст (−5 и 150 лет) — ошибки данных
Пользователь 9001	Не указана группа теста — отнести к A или B нельзя
Платёж 99003	Пустой user_id — не привязывается к пользователю
Платёж 99002	Группа в «Платежах» (B) не совпадает с группой пользователя (A)

Оставлены как валидные: возраст старше 58 лет (реальная аудитория), нетипичные ID (user_id 9004, payment_id 99001–99002), 6 переходов «открытие → ввод данных» длительностью 10–13 минут, платёж 5265 (пересечение по времени с предыдущим платежом пользователя 1152; на пользовательскую метрику не влияет).

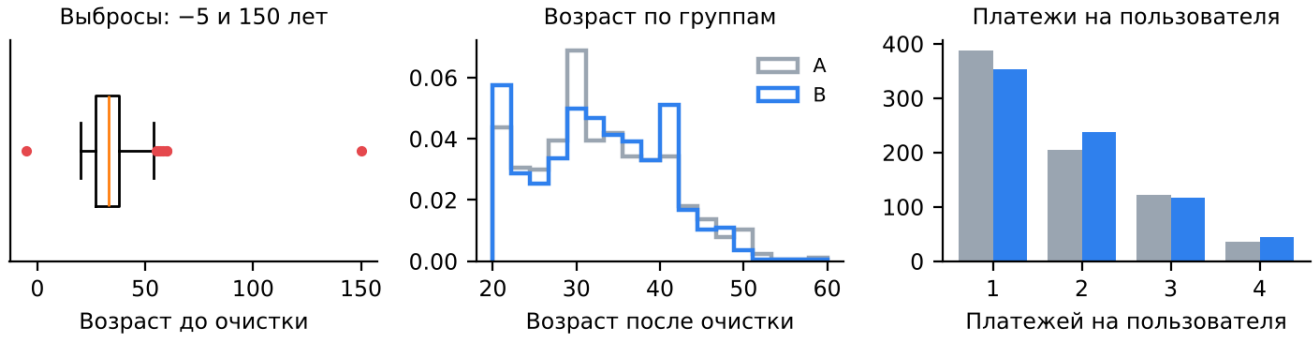


Рис. 1. Выбросы возраста до очистки, состав групп по возрасту и число платежей на пользователя после очистки.

3. Проверка корректности воронки и валидности эксперимента

Проверка	Результат	Вывод
Пропущенные шаги воронки	Нет (нет «успеха» без «подтверждения» и т.д.)	Корректно
Хронология шагов	Отрицательных и нулевых интервалов нет	Корректно
Период дат	С 1 по 30 апреля 2024	Корректно
SRM (размеры групп)	751 и 750, p = 0,98	Рандомизация не нарушена
Баланс по городу	p = 0,44	Группы сопоставимы
Баланс по устройству	p = 0,06	Граничный случай, эффект проверен по сегментам

4. Основная метрика: конверсия в успешный платёж

Группа	Пользователей	С успешным платежом	Конверсия	95% ДИ (Уилсон)
A (контроль)	751	493	**65,6%**	62,2% – 69,0%
B (тест)	750	603	**80,4%**	77,4% – 83,1%
Разница B − A			**+14,8 п.п.**	10,3 – 19,2 п.п.

z-тест: $z = 6{,}44$, $p < 0{,}001 < 0{,}05$ — H0 отвергаем. Доверительный интервал разницы не включает ноль и лежит целиком в области роста.

Проверка устойчивости: хи-квадрат $p < 0{,}001$; бутстрэп-ДИ разницы [10,4; 19,2] п.п.; логистическая регрессия с поправкой на возраст, город и устройство: отношение шансов 2,14 (95% ДИ 1,69 – 2,71). Результат не зависит от метода и состава групп.

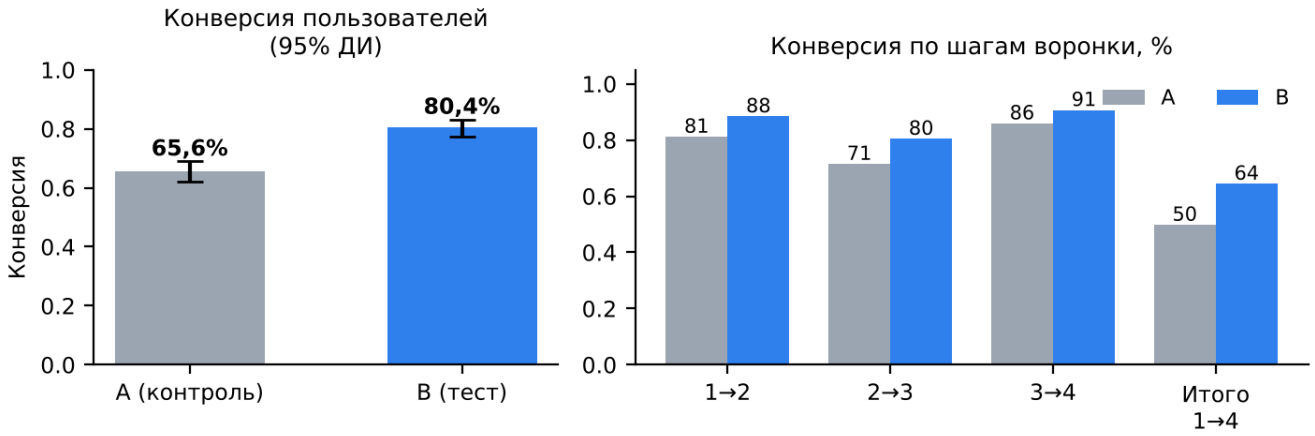


Рис. 2. Конверсия пользователей в успешный платёж по группам (слева) и конверсия на каждом шаге воронки (справа).

5. Воронка по шагам

Эффект не локален, а проходит через всю воронку (уровень платежей; тесты вторичные).

Переход	A	B	Разница	p-value
Открытие → ввод данных	81,1%	88,4%	+7,3 п.п.	< 0,001
Ввод данных → подтверждение	71,4%	80,4%	+9,0 п.п.	< 0,001
Подтверждение → успех	85,9%	90,6%	+4,7 п.п.	0,002
Весь путь: открытие → успех	**49,8%**	**64,4%**	**+14,6 п.п.**	

Сильнее всего вырос переход «ввод данных → подтверждение».

6. Защитная метрика и сегменты

Время прохождения воронки (успешные платежи): медиана 12 мин в обеих группах (A: среднее 12,4 мин, B: 12,4 мин), тест Манна — Уитни $p = 0{,}43$. Рост конверсии не достигнут ценой замедления.

Сегмент	A	B	Разница
Устройство: android	64,4%	79,9%	+15,6 п.п.
Устройство: ios	67,2%	80,8%	+13,7 п.п.
Город: Москва	62,5%	81,0%	+18,5 п.п.
Город: СПб	68,6%	84,4%	+15,9 п.п.
Город: Регион	66,8%	76,3%	+9,5 п.п.

Эффект положителен во всех сегментах; граничный дисбаланс по устройствам не объясняет результат.

7. Выводы и ограничения

- Вариант B статистически значимо лучше A: конверсия 65,6% → 80,4%, прирост +14,8 п.п.
- Результату можно доверять: рандомизация не нарушена, воронка корректна, эффект воспроизводится на всех шагах и сегментах и подтверждён четырьмя методами, защитная метрика не ухудшилась.
- **Рекомендация:** раскатить вариант B на 100% пользователей.
- **Ограничения:** тест шёл один месяц, размер выборки заранее не рассчитывался; в данных нет суммы платежей, поэтому влияние на выручку и средний чек не оценено; тесты по шагам вторичные и считались на уровне платежей (значимы и с поправкой Бонферрони).

Приложение. Полный код анализа

Код ноутбука ab-test.ipynb в порядке выполнения; читает файл case_data.xlsx (листы «Пользователи» и «Платежи»).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from statsmodels.stats.proportion import proportions_ztest, proportion_confint
```

1. Разведочный анализ данных (EDA)

```
#Посмотрим названия листов
excel_file = pd.ExcelFile('case_data.xlsx')

excel_file.sheet_names
```

1.1 Лист «Пользователи»

```
#считаем первую страницу
users = pd.read_excel(excel_file, sheet_name='Пользователи')

users.head()
print(f'Размерность датасета "Пользователи":   {users.shape}')
users.tail()
users.sample(5)
users.groupby('group').size()
users.info()
users.describe()
sns.boxplot(data=users, x='age')
users.query('age < 10')
users.query('age > 90')
users = users[~users['user_id'].isin([9002, 9003])]
users.info()
users.describe()
users.query('user_id == 9004')
users.isna().sum()
# найдем строку с na в поле group
users[users['group'].isna()]
#так как для а/б теста необходимо указание группы - удалим эту строку
users = users[users['group'].notna()]
users.groupby('group').size()
users.duplicated().sum()
users.user_id.nunique()
#Обновим знания о размерности датасета
print(f'Размерность датасета "Пользователи":   {users.shape}')
```

1.2 Лист «Платежи»

```
#Перейдем к 2 листу из файла excel
payments = pd.read_excel(excel_file, sheet_name='Платежи')
payments.head()
print(f'Размерность датасета "Платежи":   {payments.shape}')
payments.tail()
payments.sample(5)
payments.info()
payments.describe()
payments.isna().sum()
payments.nunique()
payments.duplicated().sum()
sns.boxplot(data=payments, x='payment_id')
payments.query('payment_id > 90000')
sns.boxplot(data=payments, x='user_id')
payments.query('user_id == 9004')
payments.groupby('group').size()
payments.shape
```

1.3 Объединение листов

```
df = payments.merge(users, on='user_id', how='inner') # 1 строку с nan в user_id не берем (это ~0.04% от выборки, не
    повлияет критично на результат теста)
```

```
df.head()
```

```
df.query('group_x != group_y')
```

```
df = df.query('group_x == group_y')
df = df.drop(columns='group_y')
df = df.rename(columns={'group_x': 'group'})
```

```
print(f'Размерность объединенного датасета:   {df.shape}')
```

1.4 Проверка корректности воронки

```
# список колонок с шагами воронки
steps = ['step1_opened', 'step2_entered', 'step3_confirmed', 'step4_success']
df[steps].dtypes
```

```
# пропуски по шагам (на шагах 2-4 пропуски ожидаемы - это отвалившиеся пользователи)
df[steps].isna().sum()
```

```
# шаг 1 должен быть заполнен у каждого платежа
df[df['step1_opened'].isna()]
```

```
df.query('step3_confirmed.notna() and step2_entered.isna()')
```

```
df.query('step4_success.notna() and step3_confirmed.isna()')
```

```
# воронка по количеству: каждый следующий шаг не должен быть больше предыдущего
df[steps].notna().sum()
```

```
# длительность переходов между шагами в минутах
df['t_1_2'] = (df['step2_entered'] - df['step1_opened']).dt.total_seconds() / 60
df['t_2_3'] = (df['step3_confirmed'] - df['step2_entered']).dt.total_seconds() / 60
df['t_3_4'] = (df['step4_success'] - df['step3_confirmed']).dt.total_seconds() / 60

df[['t_1_2', 't_2_3', 't_3_4']].describe()
```

```
# хронология: следующий шаг не может произойти раньше предыдущего
df.query('t_1_2 < 0')
```

```
df.query('t_2_3 < 0')
```

```
df.query('t_3_4 < 0')
```

```
# нулевые интервалы (два шага в одну и ту же минуту) - подозрительно для ручного ввода данных
df.query('t_1_2 == 0 or t_2_3 == 0 or t_3_4 == 0')
```

```
# распределение времени переходов, ищем долгие выбросы
fig, axes = plt.subplots(1, 3, figsize=(15, 4))
sns.boxplot(data=df, x='t_1_2', ax=axes[0])
sns.boxplot(data=df, x='t_2_3', ax=axes[1])
sns.boxplot(data=df, x='t_3_4', ax=axes[2])
plt.show()
```

```
# переходы, которые боксплот пометил выбросами
df.query('t_1_2 > 9')
```

```
# все даты должны попадать в период теста (апрель 2024)
print(df[steps].min())
print()
print(df[steps].max())
```

```
df = df.sort_values(['user_id', 'step1_opened'])
df['last_step_time'] = df[steps].max(axis=1)
df['prev_end'] = df.groupby('user_id')['last_step_time'].shift()

df.query('step1_opened < prev_end')
```

```
# убираем служебные колонки, они больше не нужны
df = df.drop(columns=['last_step_time', 'prev_end'])
```

2. А/Б тест

2.1 Валидность эксперимента

```
# готовим таблицу на уровне пользователя
user_stats = df.groupby('user_id').agg(n_pay=('payment_id', 'count'),
                                       n_success=('step4_success', 'count')).reset_index()

ab = users.merge(user_stats, on='user_id', how='left')
ab[['n_pay', 'n_success']] = ab[['n_pay', 'n_success']].fillna(0)
ab['success'] = ab['n_success'] > 0

ab.groupby('group').size()
```

```
# SRM: размеры групп не должны отличаться от 50/50
group_sizes = ab.groupby('group').size()
stats.chisquare(group_sizes)
```

```
# баланс групп по городу и устройству (p > 0.05 - группы сопоставимы)
print(stats.chi2_contingency(pd.crosstab(ab['group'], ab['city']))[1])
print(stats.chi2_contingency(pd.crosstab(ab['group'], ab['device_type']))[1])
```

```
fig, axes = plt.subplots(1, 4, figsize=(18, 4))

sns.histplot(data=ab, x='age', hue='group', bins=20, element='step', ax=axes[0])
axes[0].set_title('Возраст по группам')

sns.countplot(data=ab, x='city', hue='group', ax=axes[1])
axes[1].set_title('Город по группам')
sns.countplot(data=ab, x='device_type', hue='group', ax=axes[2])
```

```
axes[2].set_title('Устройство по группам')

sns.countplot(data=ab, x='n_pay', hue='group', ax=axes[3])
axes[3].set_title('Число платежей на пользователя')

plt.tight_layout()
plt.show()
```

2.2 Основная метрика: конверсия в успешный платёж

```
# конверсия по группам
conv = ab.groupby('group')['success'].agg(['sum', 'count', 'mean'])
conv
```

```
# z-тест для двух долей
z_stat, p_value = proportions_ztest(conv['sum'].values, conv['count'].values)
print(f'z = {z_stat:.3f}, p-value = {p_value:.2e}')
```

```
# доверительный интервал для разницы конверсий (B - A)
p_a = conv.loc['A', 'mean']
p_b = conv.loc['B', 'mean']
n_a = conv.loc['A', 'count']
n_b = conv.loc['B', 'count']

diff = p_b - p_a
se = np.sqrt(p_a * (1 - p_a) / n_a + p_b * (1 - p_b) / n_b)

print(f'Разница: {diff:.3%}')
print(f'95% ДИ: [{diff - 1.96 * se:.3%}; {diff + 1.96 * se:.3%}]')
```

```
# доверительные интервалы Уилсона для долей и график
conv_plot = conv.copy()
ci = proportion_confint(conv_plot['sum'], conv_plot['count'], method='wilson')
conv_plot['ci_low'] = ci[0]
conv_plot['ci_high'] = ci[1]
conv_plot['err_low'] = conv_plot['mean'] - conv_plot['ci_low']
conv_plot['err_high'] = conv_plot['ci_high'] - conv_plot['mean']

plt.figure(figsize=(6, 5))
plt.bar(conv_plot.index, conv_plot['mean'], yerr=[conv_plot['err_low'], conv_plot['err_high']],
        capsize=8, color=['#9aa5b1', '#2f80ed'])

# подписи над столбцами
for i, group in enumerate(conv_plot.index):
    plt.text(i, conv_plot.loc[group, 'ci_high'] + 0.01, f"{conv_plot.loc[group, 'mean']:.1%}", ha='center')

plt.ylim(0, 1)
plt.ylabel('Конверсия в успешный платёж')
plt.title('Конверсия пользователей по группам (95% ДИ)')
plt.show()
```

```
# условия применимости z-теста: не менее 10 успехов и 10 неуспехов в каждой группе
counts = pd.crosstab(ab['group'], ab['success'])
print(counts)
print('Условие выполнено:', counts.values.min() >= 10)
```

```
# хи-квадрат для таблицы 2x2 (без поправки Йейтса, эквивалент z-теста)
chi2, p_chi2, dof, expected = stats.chi2_contingency(counts, correction=False)
print(f'chi2 = {chi2:.2f}, p-value = {p_chi2:.2e}')
```

```
# бутстрэп разницы конверсий (B - A): 10 000 повторных выборок пользователей
np.random.seed(42)
succ_a = ab[ab['group'] == 'A']['success'].values.astype(int)
succ_b = ab[ab['group'] == 'B']['success'].values.astype(int)

boot_a = np.random.choice(succ_a, size=(10000, len(succ_a))).mean(axis=1)
boot_b = np.random.choice(succ_b, size=(10000, len(succ_b))).mean(axis=1)
boot_diff = boot_b - boot_a

print(f'Средняя разница: {boot_diff.mean():.3%}')
print(f'95% бутстрэп-ДИ: [{np.percentile(boot_diff, 2.5):.3%}; {np.percentile(boot_diff, 97.5):.3%}]')
```

```
# логистическая регрессия: эффект группы B с поправкой на возраст, город и устройство
import statsmodels.formula.api as smf

ab['success_int'] = ab['success'].astype(int)
logit = smf.logit("success_int ~ C(group) + age + C(city) + C(device_type)", data=ab).fit(disp=0)

odds_ratio = np.exp(logit.params['C(group)[T.B]'])
ci_or = np.exp(logit.conf_int().loc['C(group)[T.B]'])
print(f'Отношение шансов (B vs A): {odds_ratio:.2f}, 95% ДИ [{ci_or[0]:.2f}; {ci_or[1]:.2f}]')
print(f"p-value: {logit.pvalues['C(group)[T.B]']:.2e}")
```

2.3 Воронка по шагам

```
# воронка по шагам для каждой группы (уровень платежей)
funnel = df.groupby('group')[steps].agg(lambda x: x.notna().sum())
funnel
```

```
# конверсия шаг -> следующий шаг
funnel_conv = pd.DataFrame({
    '1 -> 2': funnel['step2_entered'] / funnel['step1_opened'],
```

```
    '2 -> 3': funnel['step3_confirmed'] / funnel['step2_entered'],
    '3 -> 4': funnel['step4_success'] / funnel['step3_confirmed'],
    'итого 1 -> 4': funnel['step4_success'] / funnel['step1_opened']
})
funnel_conv
```

```
# значимость различий на каждом переходе
for a, b in [('step1_opened', 'step2_entered'),
             ('step2_entered', 'step3_confirmed'),
             ('step3_confirmed', 'step4_success')]:
    successes = funnel[b].values
    totals = funnel[a].values
    print(a, '->', b, 'p-value =', proportions_ztest(successes, totals)[1])
```

```
# переводим воронку в доли от шага 1 и приводим к длинному формату для seaborn
funnel_share = funnel.div(funnel['step1_opened'], axis=0).T.reset_index()
funnel_share = funnel_share.melt(id_vars='index', var_name='group', value_name='share')

plt.figure(figsize=(9, 5))
ax = sns.barplot(data=funnel_share, x='index', y='share', hue='group')
for container in ax.containers:
    ax.bar_label(container, fmt='%.2f')

plt.xlabel('Шаг воронки')
plt.ylabel('Доля платежей от шага 1')
plt.title('Воронка платежа по группам')
plt.show()
```

```
# конверсия шаг -> следующий шаг (без итоговой колонки)
step_conv = funnel_conv.drop(columns='итого 1 -> 4').T.reset_index()
step_conv = step_conv.melt(id_vars='index', var_name='group', value_name='conv')

plt.figure(figsize=(8, 5))
ax = sns.barplot(data=step_conv, x='index', y='conv', hue='group')
for container in ax.containers:
    ax.bar_label(container, fmt='%.2f')

plt.xlabel('Переход')
plt.ylabel('Конверсия')
plt.title('Конверсия между шагами воронки')
plt.show()
```

2.4 Защитная метрика: время прохождения воронки

```
# общее время прохождения воронки (мин) у успешных платежей
df['t_total'] = (df['step4_success'] - df['step1_opened']).dt.total_seconds() / 60

print(df.groupby('group')['t_total'].describe())
print(stats.mannwhitneyu(df[df['group'] == 'A']['t_total'].dropna(),
                         df[df['group'] == 'B']['t_total'].dropna()))
```

```
plt.figure(figsize=(8, 5))
sns.histplot(data=df.dropna(subset=['t_total']), x='t_total', hue='group',
             bins=20, stat='density', common_norm=False, element='step')

plt.xlabel('Время от открытия до успешной оплаты, мин')
plt.title('Распределение времени прохождения воронки')
plt.show()
```

2.5 Проверка по сегментам

```
# конверсия в успешный платёж по устройствам и группам
seg = ab.groupby(['device_type', 'group'])['success'].mean().reset_index()

plt.figure(figsize=(7, 5))
ax = sns.barplot(data=seg, x='device_type', y='success', hue='group')
for container in ax.containers:
    ax.bar_label(container, fmt='%.2f')

plt.xlabel('Устройство')
plt.ylabel('Конверсия в успешный платёж')
plt.title('Конверсия по устройствам и группам')
plt.show()
```

```
# конверсия по городам и группам
ab.groupby(['city', 'group'])['success'].mean().unstack()
```

3. Итоговый вывод